Module Summary: A Reproducible Data Workflow for Titanic Survival Data

Vu Tran | AI Programming Foundations Capstone | November 2026

1. Overview

For this project I built a reproducible data workflow in a Jupyter Notebook (*data_workflow.ipynb*). The notebook loads a raw CSV file, cleans it with documented functions, explores it with reusable analysis functions, and visualizes the main survival patterns. The dataset is the training file from *Titanic - Machine Learning from Disaster* (Kaggle, n.d.), available at https://www.kaggle.com/c/titanic/data. The goal was a clean, documented workflow that someone else can rerun and that later machine learning projects can build on, not a predictive model.

2. Dataset Description

The dataset lists passengers of the RMS Titanic, together with whether each passenger survived the 1912 sinking. The file has 891 rows (one per passenger) and 12 columns, and 342 of the passengers (38.4%) survived. The columns cover identity (*PassengerId*, *Name*, *Ticket*), demographics (*Sex*, *Age*), travel details (*Pclass*, *Fare*, *Cabin*, *Embarked*), relatives aboard (*SibSp*, *Parch*), and the outcome (*Survived*). I focused on the outcome and on its relationship with sex, passenger class, and age. The data quality check found missing values in *Age* (177 rows, 19.9%), *Cabin* (687 rows, 77.1%), and *Embarked* (2 rows), and it found no duplicate rows.

3. Workflow Description

The notebook runs from top to bottom in five stages, each under its own Markdown heading:

- **Ingestion.** The notebook reads *train.csv* with pandas, displays the shape and first rows, and runs a data quality check that reports data types, duplicate rows, and missing values for each column.
- **Cleaning.** Three single-purpose functions, each with a docstring, handle the cleaning: *standardize_column_names()*, *impute_age()*, and *handle_cabin_and_embarked()*. A pipeline function, *clean_titanic()*, chains them together. Assertions then confirm that no rows were dropped and that no missing values remain in *age* or *embarked*. The raw DataFrame is never modified.

- **Exploratory analysis.** Three functions with docstrings (*summarize_numeric()*, *survival_rate_by()*, and *correlation_with_target()*) produce summary statistics, survival rates by group, and correlations with the outcome.
- **Visualizations.** The notebook creates three Seaborn/Matplotlib figures, each with a title and labeled axes, and saves them to a *figures/* folder.
- **Summary.** The notebook ends with a Markdown section that interprets the results and states the limitations and assumptions.

This structure follows Danchev (2022), who presents reproducible data science as an analysis carried out with open tools (Python, Jupyter Notebook, Markdown, and version control) and written as clean, reusable code, so that anyone can rerun the full analysis. Splitting the cleaning into small functions and combining them in one pipeline also follows the recommendation to decompose programs into functions and to record every step used to process data (Wilson et al., 2017).

4. Key Decisions and Assumptions

Cleaning choices

- **Keep the raw data unchanged.** Every cleaning function works on a copy, and *train.csv* is never edited. Wilson et al. (2017) list saving the raw data as a core practice. Keeping the original file means every number in this report can be regenerated from it.
- **Standardize column names** to lowercase snake_case (for example, *SibSp* becomes *sib_sp*). The issue addressed was inconsistent capitalization, which makes code harder to read and more error-prone to write.
- **Impute *age* with the median for the same sex and class**, after first adding an *age_was_missing* flag. The issue was 177 missing ages, which is too many rows to drop without biasing the sample. One overall median would ignore large differences between groups: the median observed age ranges from 21.5 years (third-class women) to 40 years (first-class men). The flag keeps filled-in values separate from observed ones. This matters because the bias caused by missing data depends on why the data are missing, and because a single imputed value understates uncertainty (Sterne et al., 2009).
- **Replace *cabin* with a *has_cabin* flag** and drop the original column. With 77.1% of values missing, any filled-in cabin would be mostly invented, so I kept only whether a cabin was recorded.

- **Fill the 2 missing *embarked* values with the most common port** (Southampton, 'S'). Two rows out of 891 cannot change any group-level result, and filling them avoids errors in later steps.

EDA focus

The EDA focused on survival rates by sex, by class, and by sex and class together, since those were the factors most likely to matter. I also checked whether a missing age was itself linked to survival, because that decides whether the imputation could bias the results. The correlation function screened all numeric features at once, and a separate check counted the 15 passengers with a fare of 0.

What each plot was designed to show

- **Figure 1** (grouped bar chart) shows how sex and class combine to affect survival, which neither single-factor table shows.
- **Figure 2** (stacked histogram) shows how survival changes across ages. It uses observed ages only (n = 714), so the imputed medians do not create artificial spikes.
- **Figure 3** (correlation heatmap) gives an overview of which numeric features are associated with survival and which features overlap with each other.

5. Results and Interpretation

Sex was the strongest single factor. 74.2% of women survived compared with 18.9% of men, and *is_female* had the highest correlation with survival (0.543). Survival also fell by class, from 63.0% in first class to 47.3% in second and 24.2% in third. **Figure 1** shows that the two factors interact. 96.8% of first-class and 92.1% of second-class women survived, but only 50.0% of third-class women did. Second-class and third-class men had the lowest survival rates (15.7% and 13.5%).

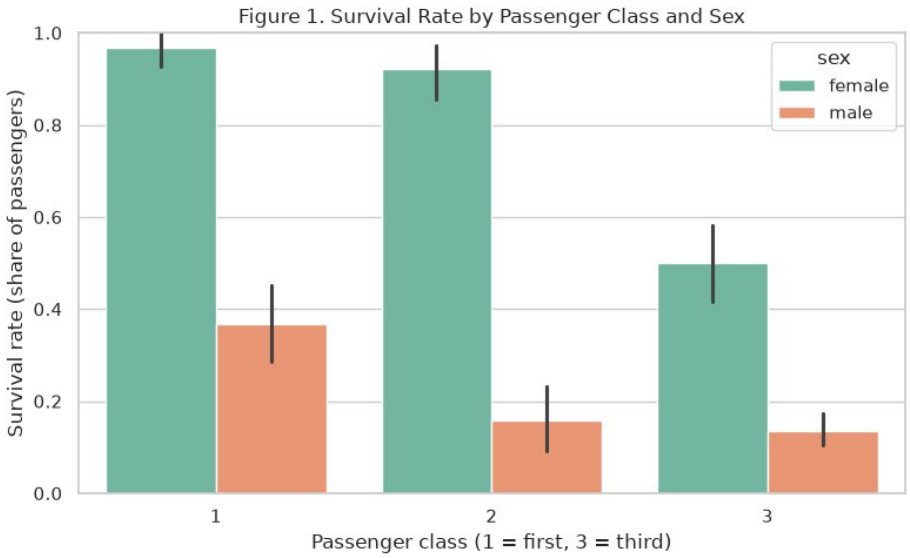


Figure 1. Survival rate by passenger class and sex. Error bars show 95% confidence intervals.

Figure 2 shows that the 0–5 age group is the only well-populated bin where survivors outnumber non-survivors. This is consistent with the 'women and children first' account. Because the advantage is concentrated among very young children, age on its own has almost no *linear* correlation with survival (-0.060). This shows why the distribution needed to be plotted instead of relying on the correlation alone.

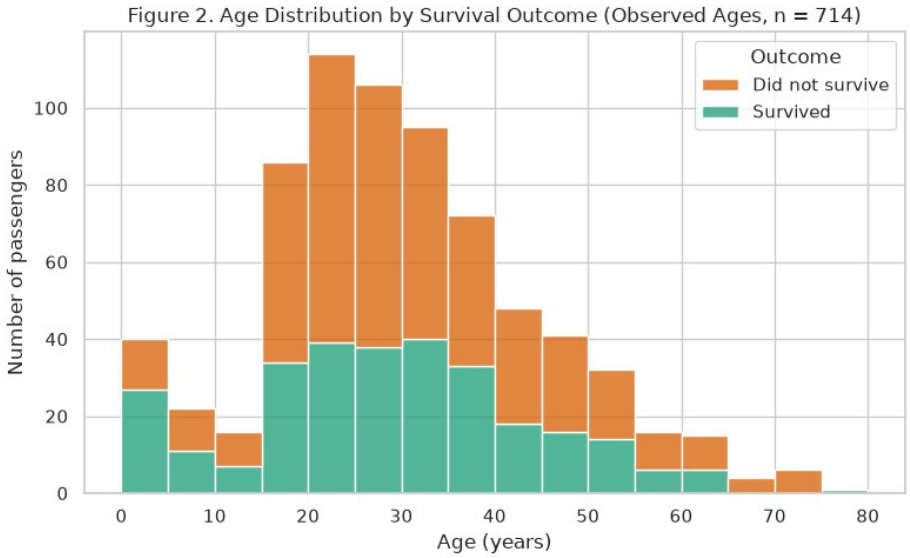


Figure 2. Age distribution of survivors and non-survivors, observed ages only (n = 714).

Figure 3 confirms *is_female* (0.54), *pclass* (-0.34), *has_cabin* (0.32), and *fare* (0.26) as the features most associated with survival. It also shows that several of these features overlap. *has_cabin* correlates -0.73 with *pclass*, and *fare* correlates -0.55 with *pclass*, so all three partly measure the same thing: the passenger's ticket class and wealth.

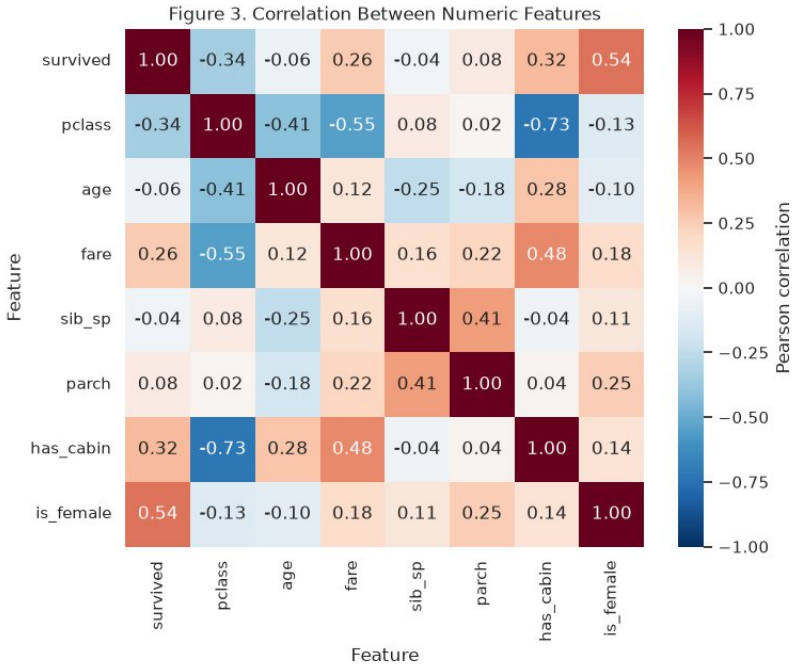


Figure 3. Pearson correlation matrix of numeric features in the cleaned data (sex encoded as is_female).

6. Responsible Practice (Bias and Data Quality)

The largest bias risk in this dataset is missing age. Missing ages are not random. They are concentrated in third class (27.7% missing, compared with 13.9% in first class and 6.0% in second), and passengers with no recorded age survived at only 29.4%, compared with 40.6% for passengers with a recorded age. Dropping those 177 rows would have removed passengers who were disproportionately poorer and less likely to survive. The remaining sample would then look wealthier and its survival rate higher than the real one. Sterne et al. (2009) describe this risk, noting that the bias from missing data depends on the reasons the values are missing. Median imputation has its own cost: it narrowed the spread of ages (the standard deviation fell from 14.5 to 13.3 years). To reduce both risks, I kept every passenger, imputed within sex-and-class groups, flagged every imputed value, and used only observed ages in Figure 2.

Two other data-quality issues could mislead an analysis. First, *has_cabin* mostly measures class rather than location on the ship, because cabin numbers are missing for 97.6% of third-class passengers but only 18.5% of first-class passengers. Second, 15 passengers have a fare of 0, and the data does not say why. I left them unchanged and flagged them rather than guess. As a next step, I would compare results with and without imputation (a sensitivity analysis). For any future model, I would report performance separately for each sex and class group, so that historical inequalities are not mistaken for predictive accuracy.

7. Reproducibility

To rerun this work, someone can clone the GitHub repository *ai-programming-foundations-project*, create a virtual environment, and run *pip install -r requirements.txt*. The *requirements.txt* file was generated with *pip freeze* from the project's virtual environment, and it pins the exact versions used, including pandas 3.1.1, NumPy 2.5.4, Matplotlib 3.11.2, and seaborn 0.13.2. They then open *data_workflow.ipynb* and choose *Restart Kernel and Run All*. The raw *train.csv* is stored next to the notebook, and the assertions stop the run if the cleaned data does not meet expectations. For version control, I developed the work on a *dev* branch, with a separate commit for each task (data, notebook setup, cleaning, EDA, visualizations, summary, README and requirements, report), and merged it into *main*, so the Git history shows how the workflow was built. This setup follows Wilson et al.'s (2017) recommendations to save the raw data, make dependencies explicit, and use a version control system.

For the final changes, I followed a feature-branch workflow: I made the edits on a separate branch (feature/report-git-workflow), merged it into dev through a pull request, and then merged dev into main through a second pull request, so the final changes reached main through reviewed pull requests.

8. Future Integration Reflections

(1) To prepare this data for a machine learning model, I would split it into training and test sets *before* imputing, and fit the sex-and-class age medians on the training set only, inside a scikit-learn pipeline, so that no information leaks from the test set. I would also one-hot encode *sex* and *embarked* and drop identifiers such as *name*, *ticket*, and *passenger_id*. (2) Before training a neural network, I would scale *age*, log-transform the heavily skewed *fare* (skew 4.79), and check the class balance (38.4% survived). I would also treat overfitting as the main risk, because 891 rows is a very small dataset for a neural network. (3) Agentic automation could help by rerunning the notebook on every commit, checking the assertions, and regenerating the figures and *requirements.txt*. It would be risky if an agent could change imputation or filtering rules without human review, because silently dropping the 177 rows with missing ages would bias the results toward wealthier passengers.

9. References

Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education, 5*(56), 156. https://doi.org/10.21105/jose.00156

Kaggle. (n.d.). *Titanic - Machine learning from disaster* [Data set]. Retrieved November 8, 2026, from https://www.kaggle.com/c/titanic/data

Sterne, J. A. C., White, I. R., Carlin, J. B., Spratt, M., Royston, P., Kenward, M. G., Wood, A. M., & Carpenter, J. R. (2009). Multiple imputation for missing data in epidemiological and clinical research: Potential and pitfalls. *BMJ*, *338*, b2393. https://doi.org/10.1136/bmj.b2393

Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, *13*(6), e1005510. https://doi.org/10.1371/journal.pcbi.1005510